Prácticas guiadas: Protección de Datos y Manejo de Estado

Materia: Ciberseguridad Aplicada · Ingeniería de Software · ITSON
Stack: Java 17 · Maven · Spring Boot · Apache NetBeans · H2 · jjwt

> **Uso ético:** todo lo ofensivo se ejecuta únicamente contra la aplicación que tú construyes en tu equipo (`localhost`). Nunca contra sistemas de terceros.

Índice

1. Preparación del proyecto (común)
2. Práctica 1 · Exposición de datos sensibles
3. Práctica 2 · Manejo de excepciones y no exponer Stack Trace
4. Práctica 3 · Logging seguro
5. Práctica 4 · Cifrado en reposo (AES-GCM)
6. Práctica 5 · Sesiones seguras: JWT y Cookies
7. Formato del reporte

0. Preparación del proyecto (común)

0.1 Requisitos

- JDK 17+, Apache NetBeans 20+, Maven (incluido en NetBeans), `curl.exe` (Windows 10/11 ya lo trae) y un navegador con DevTools (F12).
- Un solo proyecto contiene las 5 prácticas. Cada una expone `/labN/vuln/...` (versión vulnerable) y `/labN/safe/...` (versión mitigada).

0.2 Crear el proyecto en NetBeans

1. **File → New Project → Java with Maven → Java Application**. Nombre: `labseguridad`, Group Id: `mx.itson`, Package: `mx.itson.lab`.
2. Reemplaza el contenido de `pom.xml` por el siguiente.
3. Crea (**New → Java Class**) las clases indicadas más abajo, todas en el paquete `mx.itson.lab`. Borra la clase `App` o `Main` generada.

4. Crea `src/main/resources/application.properties` y `src/main/resources/schema.sql` (**New → Other → Other → Properties File / Empty File**).

pom.xml (ajusta la versión de Spring Boot a la última estable de la serie 3.x)

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSche
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/m
  <modelVersion>4.0.0</modelVersion>
  <groupId>mx.itson</groupId>
  <artifactId>labseguridad</artifactId>
  <version>1.0</version>
  <packaging>jar</packaging>
  <name>labseguridad</name>
  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <maven.compiler.release>17</maven.compiler.release>
    <maven.compiler.parameters>true</maven.compiler.parameters>
    <spring-boot.version>3.5.0</spring-boot.version>
    <jjwt.version>0.12.6</jjwt.version>
  </properties>
  <dependencyManagement>
    <dependencies>
      <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-dependencies</artifactId>
        <version>${spring-boot.version}</version>
        <type>pom</type>
        <scope>import</scope>
      </dependency>
    </dependencies>
  </dependencyManagement>
  <dependencies>
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-w
    <dependency><groupId>org.springframework.boot</groupId><artifactId>spring-boot-starter-d
    <dependency><groupId>com.h2database</groupId><artifactId>h2</artifactId><version>2.3.232
    <dependency><groupId>org.springframework.security</groupId><artifactId>spring-security-c
    <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-api</artifactId><version>$
    <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-impl</artifactId><version
    <dependency><groupId>io.jsonwebtoken</groupId><artifactId>jjwt-jackson</artifactId><vers
  </dependencies>
  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
        <version>${spring-boot.version}</version>
```

```
        <configuration><mainClass>mx.itson.lab.LabApplication</mainClass></configuration>
        <executions><execution><goals><goal>repackage</goal></goals></execution></executions
      </plugin>
    </plugins>
  </build>
</project>
```

application.properties (la configuración de errores es vulnerable a propósito; se corrige en la Práctica 2)

```
spring.datasource.url=jdbc:h2:file:./data/labdb
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false
spring.sql.init.mode=always
spring.h2.console.enabled=true

# Práctica 2: configuración vulnerable (A05)
server.error.include-stacktrace=always
server.error.include-message=always
server.error.include-exception=true
```

schema.sql

```
CREATE TABLE IF NOT EXISTS nota_plana   (id BIGINT AUTO_INCREMENT PRIMARY KEY, usuario VARCHA
CREATE TABLE IF NOT EXISTS nota_cifrada (id BIGINT AUTO_INCREMENT PRIMARY KEY, usuario VARCHA
```

Usuario.java (campos públicos solo por brevedad didáctica)

```
package mx.itson.lab;

import jakarta.persistence.*;

@Entity
public class Usuario {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) public Long id;
    public String nombre, email, passwordHash, curp, tarjeta, rol;

    public static Usuario of(String n, String e, String h, String c, String t, String r) {
        Usuario u = new Usuario();
        u.nombre = n; u.email = e; u.passwordHash = h; u.curp = c; u.tarjeta = t; u.rol = r;
        return u;
    }
}
```

UsuarioRepository.java

```
package mx.itson.lab;

import java.util.Optional;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UsuarioRepository extends JpaRepository<Usuario, Long> {
    Optional<Usuario> findByEmail(String email);
}
```

LabApplication.java (clase principal + datos de prueba)

```
package mx.itson.lab;

import org.springframework.boot.*;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;

@SpringBootApplication
public class LabApplication {
    public static void main(String[] args) { SpringApplication.run(LabApplication.class, args

    @Bean CommandLineRunner seed(UsuarioRepository r) {
        return args -> {
            if (r.count() > 0) return;
            var enc = new BCryptPasswordEncoder();
            r.save(Usuario.of("Ana Pérez", "ana@itson.mx", enc.encode("Ana-2026"),
                    "PEXA900101MSRRNN01", "4111111111111111", "USER"));
            r.save(Usuario.of("Admin", "admin@itson.mx", enc.encode("Admin-2026"),
                    "AAAA800101HSRDMN02", "5500000000000004", "ADMIN"));
        };
    }
}
```

0.3 Generar las llaves (Prácticas 4 y 5)

Necesitas dos llaves aleatorias de 256 bits en hexadecimal (64 caracteres). Elige una opción:

```
# Linux / macOS / Git Bash
openssl rand -hex 32

# PowerShell 7+
[Convert]::ToHexString([Security.Cryptography.RandomNumberGenerator]::GetBytes(32))
```

Ejecútalo dos veces: una para `LAB_AES_KEY` y otra para `LAB_JWT_KEY`. **Nunca** las escribas en el código ni las subas a Git.

0.4 Ejecutar

Opción A (terminal, recomendada), desde la carpeta del proyecto:

```
# PowerShell
$env:LAB_AES_KEY="<hex1>"; $env:LAB_JWT_KEY="<hex2>"; mvn spring-boot:run

# Bash
LAB_AES_KEY=<hex1> LAB_JWT_KEY=<hex2> mvn spring-boot:run
```

Opción B (NetBeans): clic derecho al proyecto → **Run Maven** → **Goals…** → Goals: `spring-boot:run` → **Set Properties** → agrega `spring-boot.run.jvmArguments=-DLAB_AES_KEY=<hex1> -DLAB_JWT_KEY=<hex2>`.

Las Prácticas 1 a 3 funcionan sin llaves. La app queda en `http://localhost:8080`.

Usuario	Contraseña	Rol
`ana@itson.mx`	`Ana-2026`	USER
`admin@itson.mx`	`Admin-2026`	ADMIN

> Tras cada cambio de código, detén y vuelve a ejecutar. La base de datos persiste en la carpeta `data/`; bórrala si quieres reiniciar desde cero.

Práctica 1 · Exposición de datos sensibles

Qué es

La aplicación entrega, almacena o transmite datos que el cliente no necesita ver: hashes de contraseña, CURP, número completo de tarjeta, tokens, etc.

Cómo sucede

- Se devuelve la **entidad JPA** directamente en el controlador; Jackson serializa *todos* sus campos.
- Se reutiliza el mismo modelo para BD, lógica y respuesta (sin capa DTO).

- Se piensa que "el frontend no lo muestra", pero la respuesta HTTP sí lo contiene.

Cómo se mitiga

- DTO de salida con lista blanca de campos (principio de mínimo privilegio de datos).
- Enmascarar lo que deba mostrarse parcialmente (tarjeta: últimos 4 dígitos).
- No almacenar lo que no necesitas (CVV nunca; evaluar tokenización).
- TLS en tránsito (Prácticas de redes) y cifrado en reposo (Práctica 4).

Referencias: OWASP Top 10:2021 **A02 Cryptographic Failures** (antes "Sensitive Data Exposure") y A04 Insecure Design · CWE-200 · CWE-359.

Código: Lab1Datos.java

```
package mx.itson.lab;

import java.util.List;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/lab1")
public class Lab1Datos {
    public record UsuarioDto(Long id, String nombre, String email, String tarjeta) {}

    private final UsuarioRepository repo;
    public Lab1Datos(UsuarioRepository repo) { this.repo = repo; }

    // ❌ VULNERABLE: expone la entidad completa (passwordHash, curp, tarjeta completa)
    @GetMapping("/vuln/usuarios")
    public List<Usuario> vuln() { return repo.findAll(); }

    // ✅ MITIGADO: DTO con lista blanca + enmascarado
    @GetMapping("/safe/usuarios")
    public List<UsuarioDto> safe() {
        return repo.findAll().stream()
                .map(u -> new UsuarioDto(u.id, u.nombre, u.email,
                        "**** **** **** " + u.tarjeta.substring(u.tarjeta.length() - 4)))
                .toList();
    }
}
```

Guía paso a paso

1. Ejecuta la app y abre una terminal.
2. **Versión vulnerable:**

```
curl.exe http://localhost:8080/lab1/vuln/usuarios
```

Identifica qué campos sensibles aparecen. Anota: ¿qué haría un atacante con `passwordHash`? (pista: cracking offline).

3. **Versión mitigada:**

```
curl.exe http://localhost:8080/lab1/safe/usuarios
```

Compara los campos de ambas respuestas.

4. **Reto (TODO):** agrega al DTO el campo `rol` y justifica por escrito si debe o no exponerse.

> **Evidencia a documentar:** captura de ambas respuestas, tabla comparativa de campos expuestos (vuln vs safe), clasificación OWASP/CWE, y respuesta a: *¿Por qué ocultar el campo en el frontend no es una mitigación?*

Práctica 2 · Manejo de excepciones y no exponer Stack Trace

Qué es

Un error no controlado devuelve al cliente información interna: clases, rutas, versiones de librerías, consultas SQL. Es reconocimiento gratuito para un atacante.

Cómo sucede

- Excepciones sin capturar que llegan al contenedor.
- `e.printStackTrace()` o `e.getMessage()` enviados en la respuesta.
- Configuración de depuración activa en producción (`server.error.include-stacktrace=always`).

Cómo se mitiga

- Manejador global (`@RestControllerAdvice`) que responde un mensaje genérico y un **identificador de correlación**.
- El detalle completo se registra *solo en el log del servidor* con ese identificador.
- Configuración de producción sin stack trace ni mensajes (`never`).
- Fallar de forma segura: ante la duda, denegar.

Referencias: OWASP Top 10:2021 **A05 Security Misconfiguration** y A04 · CWE-209, CWE-497. (Si tu curso usa la edición 2025, verifica la numeración, ya que se agregó una categoría de manejo de condiciones excepcionales.)

Código: Lab2Excepciones.java

```
package mx.itson.lab;

import java.util.*;
import org.slf4j.*;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.method.annotation.MethodArgumentTypeMismatchException;

public class Lab2Excepciones {

    // ❌ VULNERABLE: la excepción NoSuchElementException no se maneja
    @RestController @RequestMapping("/lab2/vuln")
    public static class VulnCtl {
        private final UsuarioRepository repo;
        public VulnCtl(UsuarioRepository repo) { this.repo = repo; }

        @GetMapping("/usuario/{id}")
        public Usuario get(@PathVariable Long id) { return repo.findById(id).get(); }
    }

    // ✅ MITIGADO: el controlador lanza, el Advice responde de forma genérica
    @RestController @RequestMapping("/lab2/safe")
    public static class SafeCtl {
        private final UsuarioRepository repo;
        public SafeCtl(UsuarioRepository repo) { this.repo = repo; }

        @GetMapping("/usuario/{id}")
        public Map<String, String> get(@PathVariable Long id) {
            Usuario u = repo.findById(id).orElseThrow(NoSuchElementException::new);
            return Map.of("nombre", u.nombre);
        }
    }

    @RestControllerAdvice(assignableTypes = SafeCtl.class)
    public static class Advice {
        private static final Logger log = LoggerFactory.getLogger(Advice.class);

        @ExceptionHandler(NoSuchElementException.class)
        public ResponseEntity<Map<String, String>> noEncontrado(NoSuchElementException e) {
            return ResponseEntity.status(404).body(Map.of("error", "Recurso no encontrado"));
        }

        @ExceptionHandler(MethodArgumentTypeMismatchException.class)
```

```
        public ResponseEntity<Map<String, String>> invalido(MethodArgumentTypeMismatchExcept
            return ResponseEntity.status(400).body(Map.of("error", "Solicitud inválida"));
        }

        @ExceptionHandler(Exception.class)
        public ResponseEntity<Map<String, String>> interno(Exception e) {
            String ref = UUID.randomUUID().toString().substring(0, 8);
            log.error("ref={} error interno no controlado", ref, e);   // detalle solo en el
            return ResponseEntity.status(500).body(Map.of("error", "Error interno", "ref", r
        }
    }
}
```

Guía paso a paso

1. **Vulnerable, recurso inexistente:**

   ```
   curl.exe -i http://localhost:8080/lab2/vuln/usuario/999
   ```

2. **Vulnerable, tipo inválido:**

   ```
   curl.exe -i http://localhost:8080/lab2/vuln/usuario/abc
   ```

   Localiza en la respuesta los campos `exception`, `message` y `trace`. Anota paquetes, clases y librerías que quedan expuestas.
3. **Mitigado (código):**

   ```
   curl.exe -i http://localhost:8080/lab2/safe/usuario/999
   curl.exe -i http://localhost:8080/lab2/safe/usuario/abc
   curl.exe -i http://localhost:8080/lab2/safe/usuario/1
   ```

   Compara códigos HTTP y cuerpo.
4. **Mitigado (configuración):** en `application.properties` cambia a:

   ```
   server.error.include-stacktrace=never
   server.error.include-message=never
   server.error.include-exception=false
   ```

   Reinicia y repite el paso 1. ¿Qué cambió? Esta es la defensa en profundidad para errores que el Advice no cubre.
5. **Forzar un 500 controlado:** en `SafeCtl` lanza temporalmente `throw new IllegalStateException("fallo de prueba")`, llama el endpoint y busca en la consola del servidor la línea con el `ref` recibido.

Evidencia a documentar: capturas de la respuesta vulnerable (con trace) y de la mitigada, lista de información sensible filtrada en la vulnerable, captura del log del servidor con el `ref`, y respuesta a: *¿Por qué el mismo ID de correlación debe estar en la respuesta y en el log?*

Práctica 3 · Logging seguro

Qué es

Registrar eventos útiles para detección y auditoría sin filtrar secretos y sin permitir que el atacante manipule el contenido del log.

Cómo sucede

- Se registran contraseñas, tokens, tarjetas o datos personales ("para depurar").
- **Log injection:** se registra entrada del usuario sin sanear; un salto de línea (`\r\n`) permite falsificar entradas del log.
- Los logs se almacenan con menos protección que la base de datos y los leen muchas personas y herramientas.

Cómo se mitiga

- Nunca registrar secretos; enmascarar identificadores.
- Sanear/neutralizar saltos de línea y limitar longitud de datos de usuario.
- Logs estructurados con campos estables (`evento=...`), nivel adecuado y marca de tiempo.
- Registrar eventos de seguridad (login fallido, cambios de privilegio) y monitorearlos.

Referencias: OWASP Top 10:2021 **A09 Security Logging and Monitoring Failures** · CWE-532 (info sensible en logs) · CWE-117 (log injection).

Código: Lab3Logging.java

```
package mx.itson.lab;

import org.slf4j.*;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/lab3")
```

```java
public class Lab3Logging {
    private static final Logger log = LoggerFactory.getLogger(Lab3Logging.class);
    private final UsuarioRepository repo;
    private final BCryptPasswordEncoder enc = new BCryptPasswordEncoder();
    public Lab3Logging(UsuarioRepository repo) { this.repo = repo; }

    // ❌ VULNERABLE: registra la contraseña y la entrada del usuario sin sanear
    @PostMapping("/vuln/login")
    public String vuln(@RequestParam String email, @RequestParam String password) {
        log.info("Intento de login email={} password={}", email, password);
        boolean ok = repo.findByEmail(email).filter(u -> enc.matches(password, u.passwordHas
        log.info("Resultado login {} para {}", ok ? "EXITOSO" : "FALLIDO", email);
        return ok ? "OK" : "Credenciales inválidas";
    }

    // ✅ MITIGADO: sin secretos, entrada saneada, correo enmascarado, formato estructurado
    @PostMapping("/safe/login")
    public String safe(@RequestParam String email, @RequestParam String password) {
        boolean ok = repo.findByEmail(email).filter(u -> enc.matches(password, u.passwordHas
        String e = enmascarar(limpiar(email));
        if (ok) log.info("evento=LOGIN resultado=EXITOSO email={}", e);
        else    log.warn("evento=LOGIN resultado=FALLIDO email={}", e);
        return ok ? "OK" : "Credenciales inválidas";
    }

    static String limpiar(String s) {
        String t = s.replaceAll("[\\r\\n\\t]", "_");        // neutraliza CR/LF (CWE-117)
        return t.length() > 60 ? t.substring(0, 60) : t;     // limita longitud
    }

    static String enmascarar(String correo) {
        return correo.replaceAll("(?<=.{2}).(?=[^@]*@)", "*"); // an***@itson.mx
    }
}
```

Guía paso a paso

1. **Vulnerable, login normal** (observa la consola de NetBeans/terminal donde corre la app):

   ```
   curl.exe -X POST http://localhost:8080/lab3/vuln/login -d "email=ana@itson.mx&password=
   ```

   Localiza la contraseña en texto claro en el log.
2. **Vulnerable, log injection** (inyecta un salto de línea y una línea falsa de éxito para el administrador):

```
curl.exe -X POST http://localhost:8080/lab3/vuln/login -d "email=x@x.mx%0a2026-10-26 12
```

   En el log aparecen **dos** líneas aunque solo hubo un intento. Explica cómo esto engaña a un analista o a un SIEM.
3. **Mitigado:** repite ambos comandos cambiando `/vuln/` por `/safe/`. Verifica que no hay contraseña, que el correo está enmascarado y que el salto de línea se convirtió en `_`.
4. **Reto (TODO):** agrega un contador de intentos fallidos por correo y emite `log.warn` con `evento=POSIBLE_FUERZA_BRUTA` al llegar a 5 (sin registrar la contraseña).

> **Evidencia a documentar:** fragmentos de log vulnerable y mitigado (resalta las diferencias), explicación de CWE-532 y CWE-117, y respuesta a: *¿Qué datos nunca deben ir a un log y cuáles sí son útiles para un análisis forense?*

Práctica 4 · Criptografía práctica: cifrado en reposo

Qué es

Proteger los datos almacenados (BD, archivos, respaldos) para que, si alguien obtiene el medio de almacenamiento, no pueda leerlos sin la llave.

Cómo sucede el problema

- Datos sensibles en texto plano en la BD, respaldos o discos.
- Confundir **codificación** (Base64) con **cifrado**.
- Llaves hardcodeadas en el código o en el repositorio; modos inseguros (ECB); IV fijo o reutilizado.

Cómo se mitiga

- Cifrado autenticado: **AES-256-GCM** (confidencialidad + integridad) con un **IV aleatorio de 12 bytes por registro**.
- La llave vive **fuera del código** (variable de entorno aquí; en producción un KMS/Vault, ver Unidad 3) y se rota periódicamente.
- Usar librerías/APIs estándar; no inventar algoritmos.
- Capas complementarias: cifrado de disco (BitLocker/LUKS), TDE de la BD, respaldos cifrados.

- Contraseñas: **hash** con BCrypt/Argon2, no cifrado reversible.

Referencias: OWASP Top 10:2021 **A02 Cryptographic Failures** · CWE-311, CWE-312, CWE-321, CWE-798 · OWASP Cryptographic Storage Cheat Sheet.

Código: Lab4Cifrado.java

```
package mx.itson.lab;

import java.nio.charset.StandardCharsets;
import java.security.*;
import java.util.*;
import javax.crypto.Cipher;
import javax.crypto.spec.*;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Service;
import org.springframework.web.bind.annotation.*;

public class Lab4Cifrado {

    @Service
    public static class Crypto {
        @Value("${LAB_AES_KEY:}") private String hex;
        private final SecureRandom rnd = new SecureRandom();

        private SecretKeySpec key() {
            if (hex.length() != 64) throw new IllegalStateException("Defina LAB_AES_KEY (64 
            return new SecretKeySpec(HexFormat.of().parseHex(hex), "AES");
        }

        public String cifrar(String texto) {
            try {
                byte[] iv = new byte[12];
                rnd.nextBytes(iv);                                  // IV nuevo por registro
                Cipher c = Cipher.getInstance("AES/GCM/NoPadding");
                c.init(Cipher.ENCRYPT_MODE, key(), new GCMParameterSpec(128, iv));
                byte[] ct = c.doFinal(texto.getBytes(StandardCharsets.UTF_8));
                byte[] out = new byte[12 + ct.length];
                System.arraycopy(iv, 0, out, 0, 12);
                System.arraycopy(ct, 0, out, 12, ct.length);        // guardamos IV || cifra
                return Base64.getEncoder().encodeToString(out);
            } catch (GeneralSecurityException e) { throw new IllegalStateException(e); }
        }

        public String descifrar(String b64) {
            try {
                byte[] in = Base64.getDecoder().decode(b64);
                Cipher c = Cipher.getInstance("AES/GCM/NoPadding");
```

```
                c.init(Cipher.DECRYPT_MODE, key(), new GCMParameterSpec(128, in, 0, 12));
                return new String(c.doFinal(in, 12, in.length - 12), StandardCharsets.UTF_8);
            } catch (GeneralSecurityException e) { throw new IllegalStateException(e); }
        }
    }

    @RestController @RequestMapping("/lab4")
    public static class Ctl {
        private final JdbcTemplate jdbc;
        private final Crypto crypto;
        public Ctl(JdbcTemplate jdbc, Crypto crypto) { this.jdbc = jdbc; this.crypto = crypto

        // ❌ VULNERABLE: texto plano en la BD
        @PostMapping("/vuln/notas")
        public String guardarPlano(@RequestParam String usuario, @RequestParam String conten
            jdbc.update("INSERT INTO nota_plana(usuario, contenido) VALUES (?, ?)", usuario,
            return "guardada";
        }

        // ✅ MITIGADO: AES-256-GCM antes de persistir
        @PostMapping("/safe/notas")
        public String guardarCifrada(@RequestParam String usuario, @RequestParam String cont
            jdbc.update("INSERT INTO nota_cifrada(usuario, contenido) VALUES (?, ?)",
                    usuario, crypto.cifrar(contenido));
            return "guardada";
        }

        @GetMapping("/safe/notas")
        public List<Map<String, String>> leerCifradas() {
            return jdbc.query("SELECT usuario, contenido FROM nota_cifrada",
                    (rs, i) -> Map.of("usuario", rs.getString(1),
                                      "contenido", crypto.descifrar(rs.getString(2))));
        }
    }
}
```

Guía paso a paso

1. Ejecuta con `LAB_AES_KEY` definida (sección 0.4).
2. **Guardar la misma nota en ambas versiones:**

```
curl.exe -X POST http://localhost:8080/lab4/vuln/notas -d "usuario=ana&contenido=SECRET
curl.exe -X POST http://localhost:8080/lab4/safe/notas -d "usuario=ana&contenido=SECRET
```

3. **Ver lo que hay en disco.** Abre `http://localhost:8080/h2-console`, JDBC URL `jdbc:h2:file:./data/labdb`, usuario `sa`, contraseña vacía:

```
SELECT * FROM nota_plana;
SELECT * FROM nota_cifrada;
```

Compara. Guarda la *misma* nota otra vez en `/safe/notas` y consulta de nuevo: los textos cifrados son distintos (IV aleatorio).

4. **Lectura legítima:** `curl.exe http://localhost:8080/lab4/safe/notas` devuelve el texto descifrado.
5. **Simular robo del archivo.** Detén la app y busca el texto en el archivo de la BD:

```
# Windows
findstr /M /C:"SECRETO" data\labdb.mv.db
# Linux / macOS
grep -a -c "SECRETO" data/labdb.mv.db
```

El texto de la tabla plana debe ser localizable; el de la tabla cifrada no.

6. **Integridad (extra):** en la consola H2 altera la nota cifrada y vuelve a leerla:

```
UPDATE nota_cifrada SET contenido = 'A' || SUBSTRING(contenido, 2);
```

`GET /lab4/safe/notas` falla (el tag GCM no valida). Explica por qué esto es una ventaja frente a AES-CBC sin autenticación.

7. **Reto (TODO):** inicia la app *sin* `LAB_AES_KEY` y observa el error; luego propón cómo manejarías la rotación de llaves (pista: guardar una versión de llave junto al registro).

Evidencia a documentar: capturas de ambas tablas en H2, resultado de la búsqueda en el archivo, evidencia del IV distinto por registro, resultado de la alteración, y respuesta a: *¿Dónde guardarías la llave en producción y por qué no junto a la base de datos?*

Práctica 5 · Gestión segura de sesiones: JWT y Cookies seguras

Qué es

Después de autenticar, el servidor entrega un identificador o token que prueba la identidad en cada petición. Si se roba, falsifica o nunca expira, el atacante *es* el usuario.

Cómo sucede el problema

- Secreto de firma débil, predecible o hardcodeado (cualquiera puede **forjar** tokens).
- Tokens sin expiración (`exp`) y con datos sensibles en el payload (el payload solo está codificado en Base64, **no cifrado**).
- Cookie sin `HttpOnly` (robable con XSS), sin `Secure` (viaja por HTTP) y sin `SameSite` (expuesta a CSRF).
- Token guardado en `localStorage` y devuelto en el cuerpo de la respuesta.

Cómo se mitiga

- Llave de al menos 256 bits generada aleatoriamente y fuera del código.
- Expiración corta (`exp`), `iss` validado, claims mínimos (id y rol, nada de datos personales).
- Cookie con `HttpOnly; Secure; SameSite=Strict` , `Path` acotado y `Max-Age` .
- Logout que expira la cookie; para revocación real: lista de `jti` revocados o sesiones del lado del servidor.
- Siempre validar firma y algoritmo en el servidor; nunca confiar en el rol que declara el cliente sin verificar la firma.

Referencias: OWASP Top 10:2021 **A07 Identification and Authentication Failures** y A02 · CWE-798, CWE-1004, CWE-614, CWE-1275, CWE-613 · OWASP Session Management y JWT for Java Cheat Sheets.

Código: Lab5Sesiones.java

```
package mx.itson.lab;

import io.jsonwebtoken.*;
import io.jsonwebtoken.security.Keys;
import java.nio.charset.StandardCharsets;
import java.time.*;
import java.util.*;
import javax.crypto.SecretKey;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.*;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.stereotype.Component;
import org.springframework.web.bind.annotation.*;

public class Lab5Sesiones {

    static final String CLAVE_DEBIL = "changeit-changeit-changeit-12345";   // ❌ hardcodead

    // ❌ VULNERABLE
    @RestController @RequestMapping("/lab5/vuln")
    public static class VulnCtl {
```

```
    private final UsuarioRepository repo;
    private final BCryptPasswordEncoder enc = new BCryptPasswordEncoder();
    public VulnCtl(UsuarioRepository repo) { this.repo = repo; }

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestParam String email, @RequestParam String passw
        var u = repo.findByEmail(email).filter(x -> enc.matches(password, x.passwordHash
        if (u.isEmpty()) return ResponseEntity.status(401).body("Credenciales inválidas"
        String jwt = Jwts.builder()
                .subject(u.get().email).claim("rol", u.get().rol).claim("curp", u.get().
                .signWith(Keys.hmacShaKeyFor(CLAVE_DEBIL.getBytes(StandardCharsets.UTF_8
                .compact();
        return ResponseEntity.ok()
                .header(HttpHeaders.SET_COOKIE, "SESSION=" + jwt)  // ❌ sin HttpOnly /
                .body(Map.of("token", jwt));
    }

    @GetMapping("/admin")
    public ResponseEntity<String> admin(@RequestHeader(value = "Authorization", required
        try {
            Claims c = Jwts.parser()
                    .verifyWith(Keys.hmacShaKeyFor(CLAVE_DEBIL.getBytes(StandardCharsets
                    .build().parseSignedClaims(h.substring(7)).getPayload();
            return "ADMIN".equals(c.get("rol")) ? ResponseEntity.ok("Panel ADMIN")
                                                : ResponseEntity.status(403).body("Prohi
        } catch (Exception e) { return ResponseEntity.status(401).body("Token inválido")
    }
}

// ✅ MITIGADO
@Component
public static class JwtSvc {
    @Value("${LAB_JWT_KEY:}") private String hex;

    SecretKey key() {
        if (hex.length() != 64) throw new IllegalStateException("Defina LAB_JWT_KEY (64
        return Keys.hmacShaKeyFor(HexFormat.of().parseHex(hex));
    }

    String emitir(Usuario u) {
        Instant ahora = Instant.now();
        return Jwts.builder().issuer("lab-itson").subject(String.valueOf(u.id)).claim("r
                .issuedAt(Date.from(ahora)).expiration(Date.from(ahora.plus(Duration.ofM
                .signWith(key()).compact();
    }

    Claims validar(String token) {
        return Jwts.parser().verifyWith(key()).requireIssuer("lab-itson")
                .build().parseSignedClaims(token).getPayload();   // valida firma, exp e
    }
}
```

```
    @RestController @RequestMapping("/lab5/safe")
    public static class SafeCtl {
        private final UsuarioRepository repo;
        private final JwtSvc jwt;
        private final BCryptPasswordEncoder enc = new BCryptPasswordEncoder();
        public SafeCtl(UsuarioRepository repo, JwtSvc jwt) { this.repo = repo; this.jwt = jw

        @PostMapping("/login")
        public ResponseEntity<Map<String, String>> login(@RequestParam String email, @Reques
            var u = repo.findByEmail(email).filter(x -> enc.matches(password, x.passwordHash
            if (u.isEmpty()) return ResponseEntity.status(401).body(Map.of("error", "Credenc
            ResponseCookie ck = ResponseCookie.from("ACCESS_TOKEN", jwt.emitir(u.get()))
                    .httpOnly(true).secure(true).sameSite("Strict")
                    .path("/lab5/safe").maxAge(Duration.ofMinutes(15)).build();
            return ResponseEntity.ok().header(HttpHeaders.SET_COOKIE, ck.toString())
                    .body(Map.of("mensaje", "Sesión iniciada"));
        }

        @GetMapping("/admin")
        public ResponseEntity<String> admin(@CookieValue(name = "ACCESS_TOKEN", required = fa
            if (t == null) return ResponseEntity.status(401).body("Sin sesión");
            try {
                Claims c = jwt.validar(t);
                return "ADMIN".equals(c.get("rol", String.class)) ? ResponseEntity.ok("Panel
                                                                  : ResponseEntity.status(40
            } catch (JwtException e) { return ResponseEntity.status(401).body("Sesión inváli
        }

        @PostMapping("/logout")
        public ResponseEntity<String> logout() {
            ResponseCookie ck = ResponseCookie.from("ACCESS_TOKEN", "").httpOnly(true).secur
                    .sameSite("Strict").path("/lab5/safe").maxAge(0).build();
            return ResponseEntity.ok().header(HttpHeaders.SET_COOKIE, ck.toString()).body("S
        }
    }
}
```

Guía paso a paso

Parte A · Versión vulnerable

1. Inicia sesión como Ana y observa cabeceras y cuerpo:

```
curl.exe -i -X POST http://localhost:8080/lab5/vuln/login -d "email=ana@itson.mx&passwo
```

Anota: ¿trae `HttpOnly`, `Secure`, `SameSite`? ¿Se devuelve el token en el cuerpo?

2. **Decodifica el token** (copia el valor de `token`) en `https://jwt.io` o localmente. Identifica `rol`, `curp` y la ausencia de `exp`. Concluye: ¿el JWT es secreto o solo está firmado?
3. **Acceso legítimo denegado:**

```
curl.exe -i http://localhost:8080/lab5/vuln/admin -H "Authorization: Bearer <TOKEN_DE_A
```

   Esperado: `403`.
4. **Forja de token (solo en tu laboratorio).** Supón que el secreto se filtró en el repositorio (lo ves en `CLAVE_DEBIL`; es lo que detectaría GitLeaks). En jwt.io cambia el payload a `"rol":"ADMIN"`, escribe el secreto `changeit-changeit-changeit-12345` (algoritmo HS256) y copia el token resultante:

```
curl.exe -i http://localhost:8080/lab5/vuln/admin -H "Authorization: Bearer <TOKEN_FORJ
```

   Esperado: `200 Panel ADMIN` sin conocer la contraseña de admin.
5. **Cookie legible por JavaScript.** Abre `http://localhost:8080/lab1/safe/usuarios`, pulsa F12 → Console y ejecuta:

```
fetch('/lab5/vuln/login',{method:'POST',body:new URLSearchParams({email:'ana@itson.mx',
  .then(()=>console.log(document.cookie))
```

   Aparece `SESSION=...`: un XSS podría exfiltrarla.

Parte B · Versión mitigada (requiere `LAB_JWT_KEY`)

6. Login y revisión de la cookie:

```
curl.exe -i -X POST http://localhost:8080/lab5/safe/login -d "email=ana@itson.mx&passwo
```

   Verifica `HttpOnly; Secure; SameSite=Strict; Path=/lab5/safe; Max-Age=900` y que el cuerpo ya no trae el token.
7. Envía la cookie manualmente (curl no reenvía cookies `Secure` por HTTP en todas sus versiones):

```
curl.exe -i http://localhost:8080/lab5/safe/admin -H "Cookie: ACCESS_TOKEN=<VALOR_DE_LA
```

Con Ana: `403`. Repite con `admin@itson.mx / Admin-2026`: `200`.

8. **Intento de forja:** con jwt.io modifica el rol del token de Ana y fírmalo con `changeit-changeit-changeit-12345`. Envíalo como cookie. Esperado: `401` (la firma no valida).
9. **Cookie y JavaScript:** repite el `fetch` de la Parte A con `/lab5/safe/login`. `document.cookie` ya no muestra `ACCESS_TOKEN` gracias a `HttpOnly`. En DevTools → Application → Cookies confirma que la cookie existe y revisa sus banderas. (Chrome y Firefox aceptan cookies `Secure` en `localhost`.)
10. **Expiración (opcional):** cambia `Duration.ofMinutes(15)` a `Duration.ofSeconds(20)`, reinicia, haz login, espera 25 s y llama `/safe/admin`. Esperado: `401` por token expirado.
11. **Logout:** `curl.exe -i -X POST http://localhost:8080/lab5/safe/logout` y observa `Max-Age=0`. Reflexiona: el JWT ya emitido sigue siendo válido hasta su `exp` si alguien lo copió; ¿cómo lo resolverías?

Evidencia a documentar: cabeceras `Set-Cookie` vulnerable vs mitigada, payload decodificado, capturas de la forja exitosa (vuln) y fallida (safe), prueba de `document.cookie`, tabla de banderas de cookie (qué amenaza mitiga cada una: XSS, MITM, CSRF) y respuesta a: *¿Cookie HttpOnly o localStorage para guardar el JWT, y por qué?*

Formato del reporte

Un reporte por práctica (o uno integrado), con la siguiente estructura:

1. **Datos generales:** nombre, materia, fecha, práctica.
2. **Objetivo y marco:** categoría OWASP Top 10 y CWE relacionados.
3. **Escenario vulnerable:** comando ejecutado, captura de la evidencia, análisis de qué falla y por qué.
4. **Escenario mitigado:** cambio aplicado (fragmento de código o configuración), captura y análisis.
5. **Tabla comparativa** vulnerable vs mitigado (riesgo, control aplicado, resultado).
6. **Perspectiva Red/Blue Team:** cómo lo explota un atacante y cómo lo detecta o previene el equipo defensor.
7. **Respuesta a las preguntas de reflexión y conclusiones.**
8. **Repositorio Git** con commits separados por práctica (sin llaves ni la carpeta `data/`; agrégala a `.gitignore`).

Formato ITSON: Arial 12 justificado, títulos Arial 14 en negritas, encabezado con nombre del alumno, pie de página con fecha, materia y hora.

Práctica	Tema	OWASP Top 10:2021	Endpoints
1	Exposición de datos sensibles	A02, A04	`/lab1/{vuln|safe}/usuarios`
2	Excepciones y Stack Trace	A05, A04	`/lab2/{vuln|safe}/usuario/{id}`
3	Logging seguro	A09	`/lab3/{vuln|safe}/login`
4	Cifrado en reposo	A02	`/lab4/{vuln|safe}/notas`
5	JWT y cookies	A07, A02	`/lab5/{vuln|safe}/login`, `/admin`